Software Design Document

UniHirex

Recruitment Portal for University Students

Submitted by

Sehar Tahir
S23BINFT1M01030

Submitted to

Sir Ali Nawaz Shah

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Revision History

Date	Description	Author	Comments
20-05-2026	Version 1	Sehar Tahir	First Revision

Document Approval

Signature	Printed Name	Title	Date

Supervised by

Sir Ali Nawaz Shah

Signature _____

Summary

This Software Design Document (SDD) presents the complete design and architecture of **UniHirex - Recruitment Portal for University Students**. The document provides a detailed overview of the system structure, modules, database design, UML diagrams, APIs, security mechanisms, testing strategies, and future enhancements.

The main objective of UniHirex is to provide a centralized web-based platform that connects university students with recruiters in an efficient and organized manner. The system is designed to simplify the recruitment process by enabling students to create profiles, search for jobs, and apply online, while recruiters can manage job postings and review applications through a single platform.

The application follows a MERN-based layered architecture consisting of React.js for frontend development, Node.js and Express.js for backend services, and MongoDB for database management. The design focuses on scalability, modularity, performance, security, and user-friendly interaction.

This document also highlights important design considerations such as secure authentication using JWT, password encryption, RESTful API communication, and scalable database structure. Additionally, future enhancements including AI-based job recommendations, mobile applications, and real-time notifications are discussed to improve the overall system capabilities.

Overall, the UniHirex platform is designed to provide an effective digital recruitment solution for universities, students, and recruiters while ensuring maintainability, flexibility, and future scalability.

Contents

1. Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the architecture, design, modules, and implementation details of the system **UniHirex - Recruitment Portal for University Students**. This document provides a clear understanding of how the system is structured and how different components interact with each other.

The main goal of UniHirex is to connect university students with recruiters in an efficient and organized way. It allows students to create profiles, apply for jobs, and track applications, while recruiters can post job opportunities and manage applicants.

This document is intended for developers, project supervisors, and stakeholders to understand the system design before implementation and to ensure that all requirements are properly addressed.

1.2 Scope

UniHirex is a web-based recruitment platform designed specifically for university students and recruiters. The system provides a centralized platform where:

- Students can register, build profiles, and apply for jobs.
- Recruiters can post job openings and review applications.
- Administrators can manage users, jobs, and overall system activities.

The system eliminates the traditional manual recruitment process and replaces it with a digital, efficient, and transparent workflow. It aims to reduce communication gaps between students and companies and improve job placement opportunities for graduates.

The scope of this project includes user management, job posting, application tracking, authentication system, and an administrative dashboard.

1.3 System Overview

UniHirex is built as a full-stack web application that follows a client-server architecture. The system consists of three main user roles:

- **Students**: They can create accounts, upload profiles, browse jobs, and submit applications.
- **Recruiters**: They can post jobs, manage listings, and review student applications.
- **Admins**: They manage platform users, monitor activity, and ensure system integrity.

The system communicates through RESTful APIs, ensuring smooth interaction between frontend and backend. Data is stored in a centralized database, ensuring consistency and reliability.

1.4 Design Objectives

The main design objectives of the UniHirex system are:

- To provide a simple and user-friendly interface for students and recruiters.
- To ensure secure authentication and data protection for all users.
- To design a scalable system that can handle increasing users and job postings.
- To maintain modular architecture for easy maintenance and future enhancements.
- To enable fast and efficient job search and application processing.
- To ensure smooth communication between students and recruiters through a centralized platform.

2. System Architecture

2.1 Architecture Overview

The UniHirex system follows a **client-server architecture**, where the frontend (client) communicates with the backend (server) through RESTful APIs. The system is designed to separate the user interface, business logic, and database layers to ensure better scalability, maintainability, and performance.

The frontend handles user interaction, while the backend processes requests, applies business logic, and interacts with the database. All communication between frontend and backend is done using HTTP requests in JSON format.

This layered architecture ensures that each component operates independently, making the system easier to develop, test, and scale.

2.2 Architecture Style

UniHirex is built using the **MERN-based layered architecture approach** (MongoDB, Express.js, React.js, Node.js). The system follows a three-tier architecture:

- **Presentation Layer (Frontend)**
  Built using React.js, responsible for user interface and user experience.
- **Application Layer (Backend)**
  Built using Node.js and Express.js, handles business logic, authentication, and API processing.

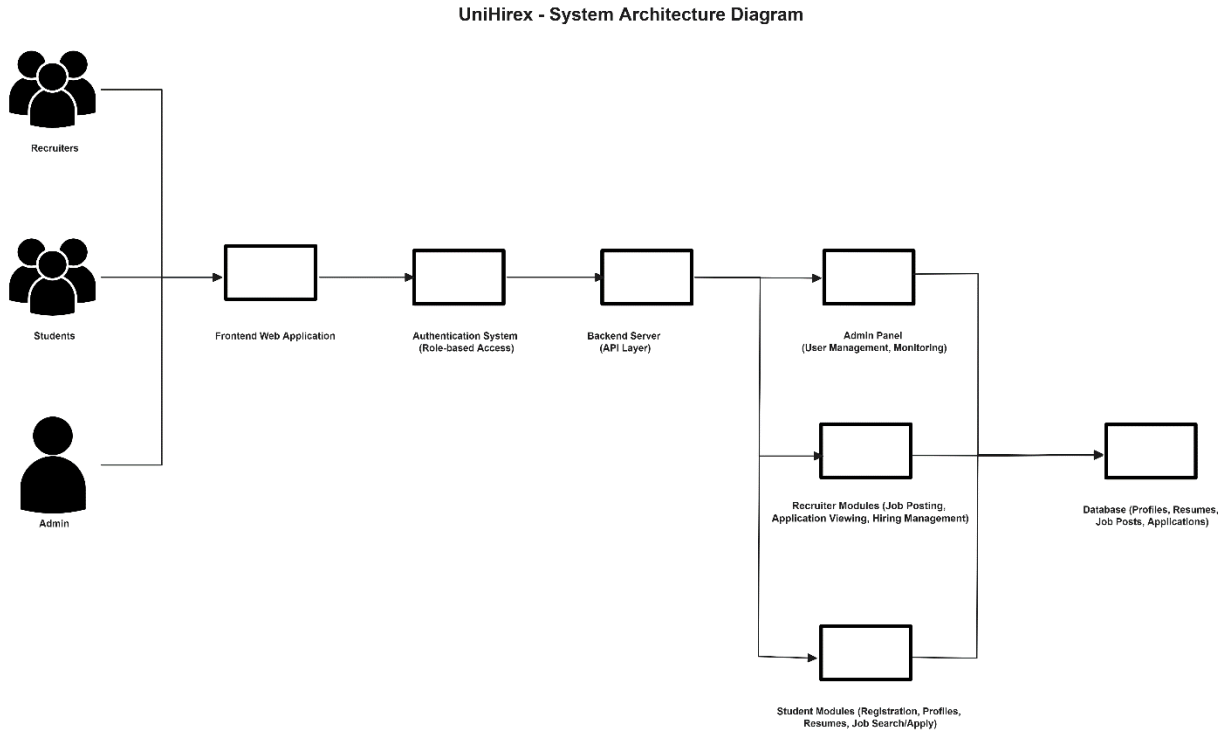
- **Data Layer (Database)**
  MongoDB is used to store and manage all system data such as users, jobs, and applications.

This architecture ensures separation of concerns and improves system flexibility.

2.3 System Architecture Diagram

The system architecture can be represented as follows:

- Users (Students, Recruiters, Admins) interact with the **Frontend (React.js)**
- Frontend sends requests to **Backend APIs (Node.js + Express.js)**
- Backend processes logic and communicates with **MongoDB Database**
- Responses are sent back to the frontend and displayed to users



2.4 Technology Stack

The UniHirex system uses the following technologies:

- **Frontend:** React.js, HTML, CSS, JavaScript
- **Backend:** Node.js, Express.js
- **Database:** MongoDB
- **Authentication:** JWT (JSON Web Tokens)

- **API Communication:** RESTful APIs
- **Version Control:** Git and GitHub

These technologies were selected to ensure scalability, performance, and ease of development.

2.5 Deployment Overview

The UniHirex system is designed to be deployed as a cloud-based web application. The deployment process includes:

- Frontend hosted on platforms like Vercel or Netlify
- Backend hosted on services like Render or AWS
- Database hosted on MongoDB Atlas (cloud database)

The system uses environment variables to manage sensitive data such as database credentials and API keys. This ensures security and flexibility across different environments (development, testing, and production).

3. Data Flow Diagrams (DFD)

3.1 DFD Level 0 (Context Diagram)

The Level 0 DFD represents the overall system as a single process and shows how external entities interact with the UniHirex system.

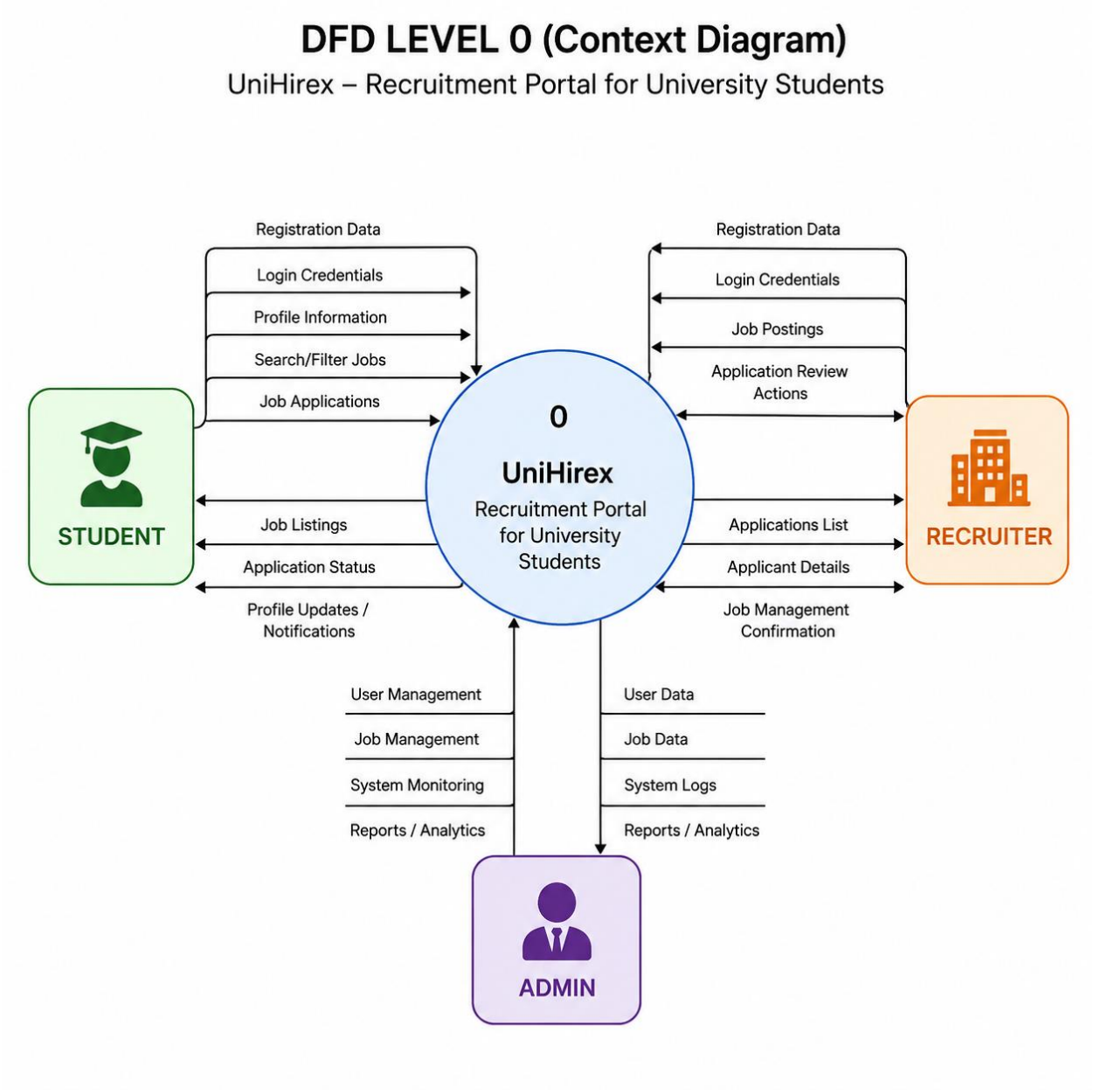
In the UniHirex system, there are three main external entities:

- **Student**
- **Recruiter**
- **Admin**

System Flow:

- Students send registration data, login requests, job applications, and profile updates to the system.
- Recruiters send job postings, login requests, and application review actions.
- Admin manages users, jobs, and system monitoring tasks.

The system processes all incoming data and returns relevant outputs such as job listings, application status, and system confirmations.



3.2 DFD Level 1

The Level 1 DFD breaks the main system into major processes. The UniHirex system consists of the following core processes:

Process 1: User Management

- Handles student, recruiter, and admin registration and login.
- Manages authentication and profile updates.

Process 2: Job Management

- Recruiters can create, update, and delete job postings.
- Students can view available job listings.

Process 3: Application Management

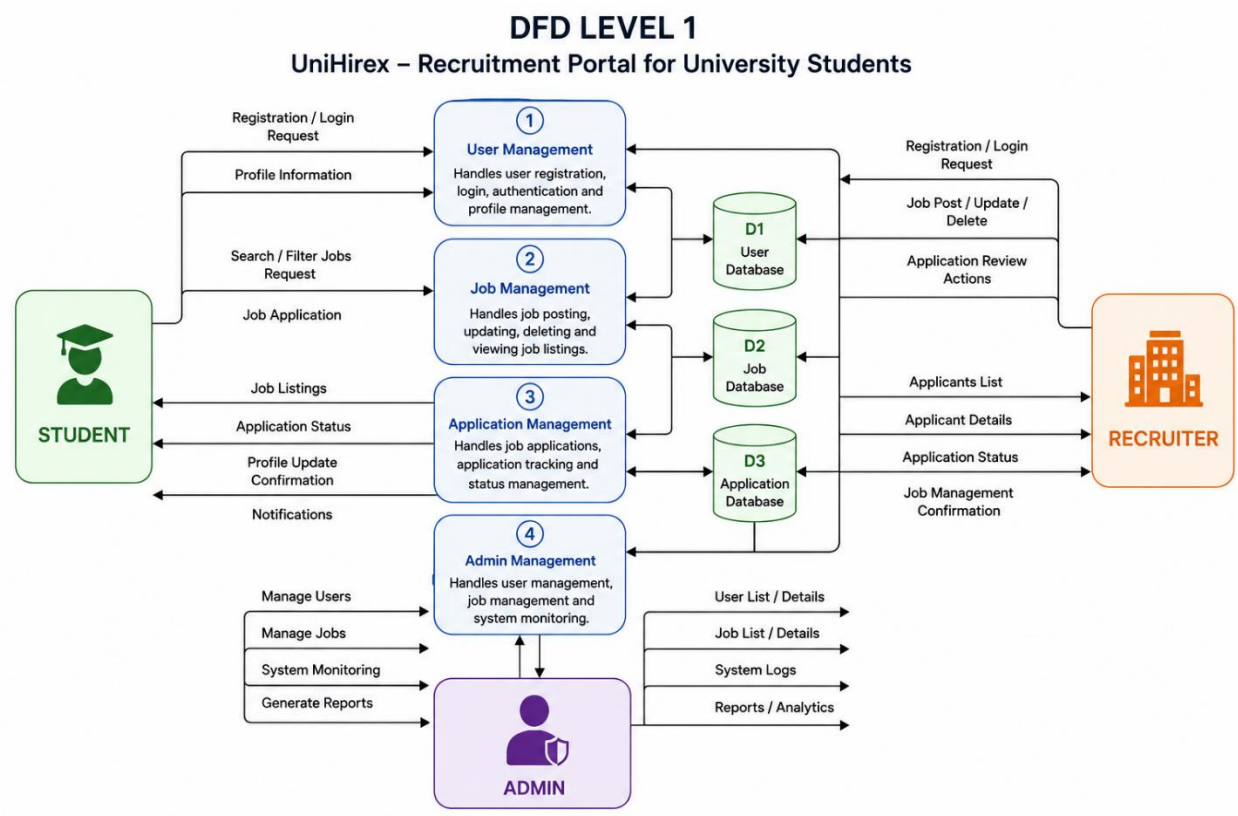
- Students apply for jobs.
- System stores and tracks applications.
- Recruiters review applications and update status.

Process 4: Admin Management

- Admin manages users and jobs.
- Admin monitors system activities.

Data Stores:

- User Database
- Job Database
- Application Database



3.3 DFD Level 2

The Level 2 DFD provides a detailed breakdown of each major process.

1. User Management (Level 2)

- Register User
- Login User
- Authenticate User (JWT)
- Update Profile

2. Job Management (Level 2)

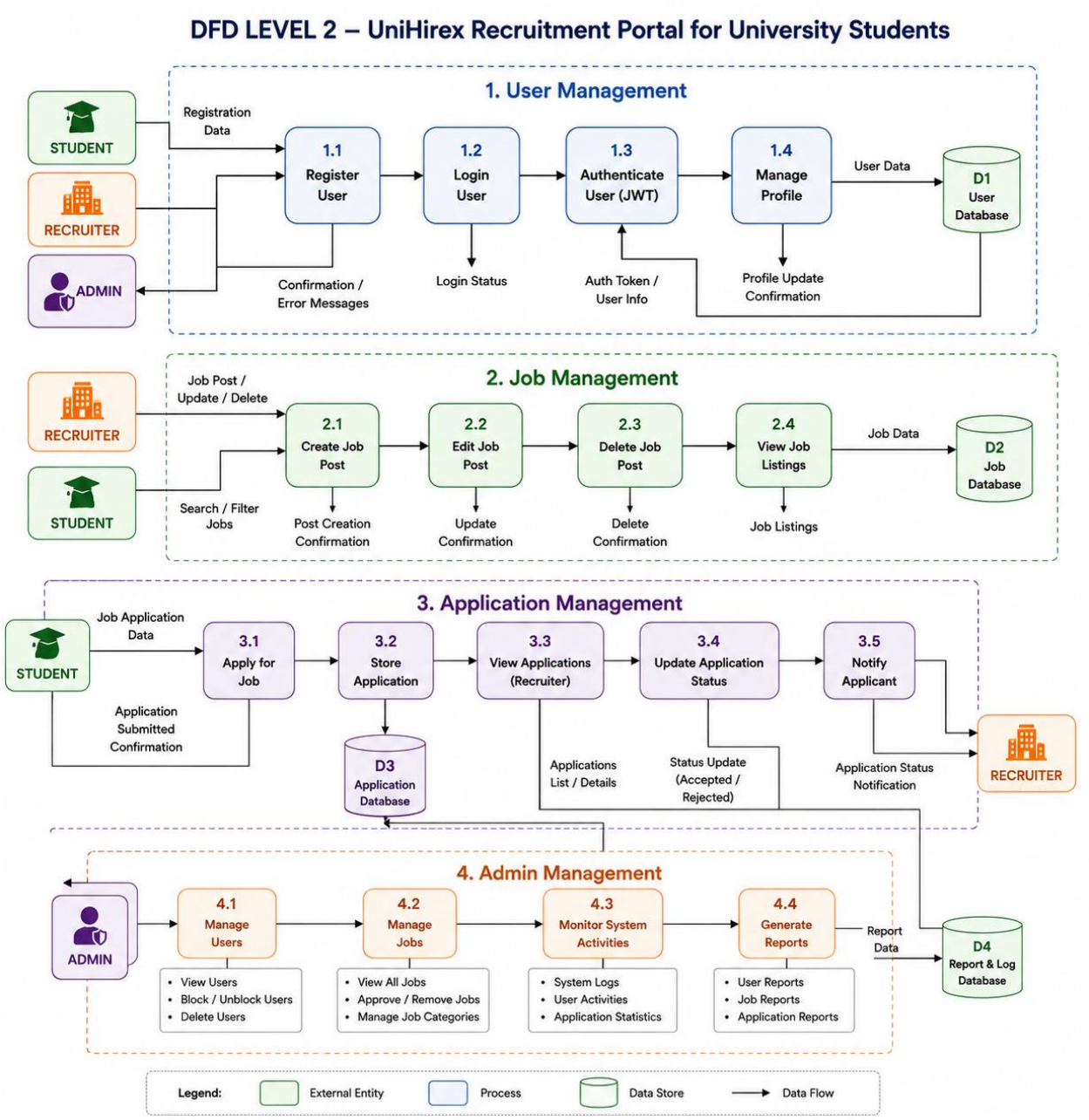
- Create Job Post (Recruiter)
- Edit Job Post
- Delete Job Post
- View Job Listings (Student)

3. Application Management (Level 2)

- Submit Job Application
- Store Application Data
- Fetch Applications for Recruiter
- Update Application Status (Accepted/Rejected)

4. Admin Management (Level 2)

- View Users
- Block/Delete Users
- Monitor Jobs
- System Oversigh



4. Database Design

4.1 Database Overview

The UniHirex system uses **MongoDB** as its primary database management system. MongoDB is a NoSQL database that stores data in the form of collections and documents, providing flexibility, scalability, and high performance for web applications.

The database is designed to manage and store all important information related to:

- Students
- Recruiters
- Admins
- Job Posts
- Job Applications
- Authentication Data

The database structure ensures efficient data retrieval, secure storage, and smooth interaction between the frontend and backend systems.

4.2 Entity Relationship Diagram (ERD)

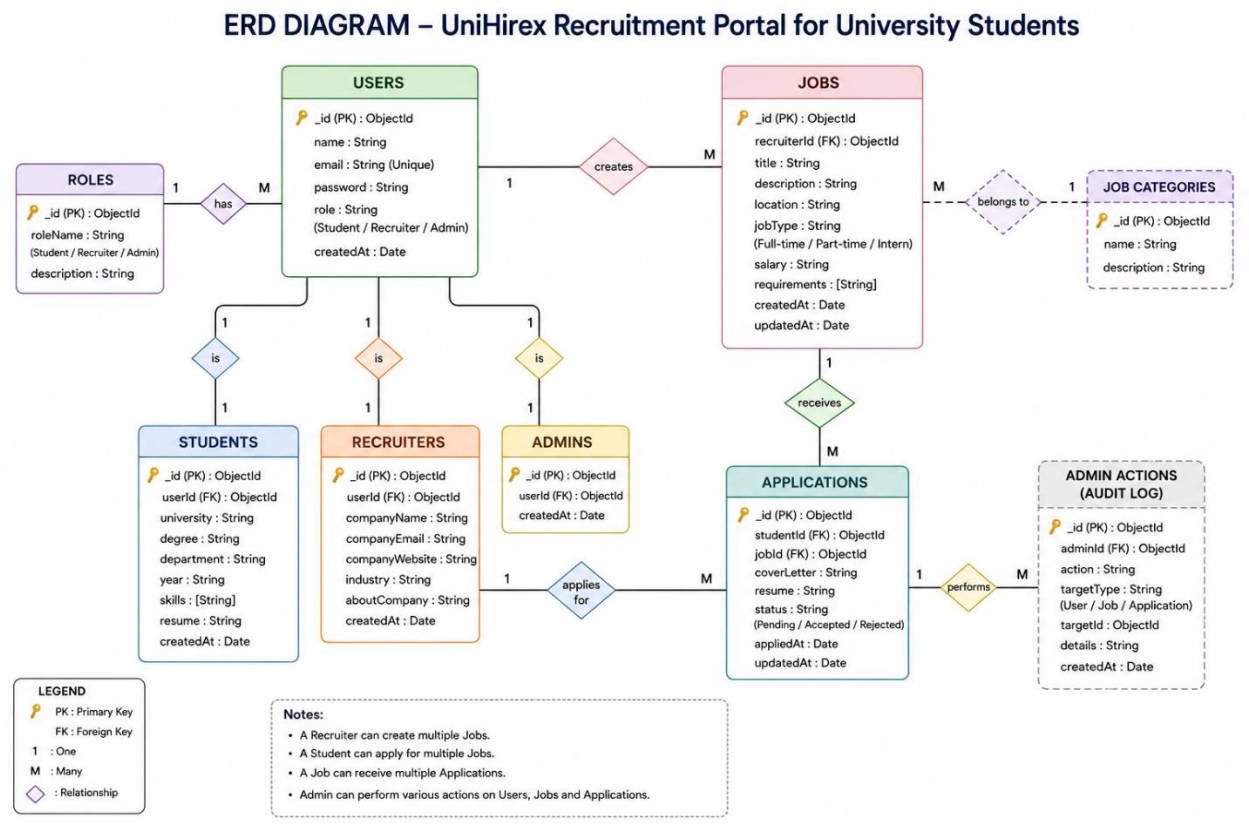
The Entity Relationship Diagram (ERD) represents the relationships between the main entities of the UniHirex system.

Main Entities

- User
- Student
- Recruiter
- Admin
- Job
- Application

Relationships

- A recruiter can create multiple jobs.
- A student can apply for multiple jobs.
- A job can receive multiple applications.
- Admin can manage all users and jobs.



4.3 Tables / Collections Description

1. Users Collection

Stores authentication and general user information.

Field Name	Data Type	Description
_id	ObjectId	Unique user ID
name	String	User full name
email	String	User email
password	String	Encrypted password
role	String	Student / Recruiter / Admin
createdAt	Date	Account creation date

2. Students Collection

Stores detailed student profile information.

Field Name	Data Type	Description
studentId	ObjectId	Unique student ID
userId	ObjectId	Reference to user

university	String	University name
degree	String	Student degree
skills	Array	Skills list
resume	String	Resume file path

3. Recruiters Collection

Stores recruiter and company information.

Field Name	Data Type	Description
recruiterId	ObjectId	Unique recruiter ID
userId	ObjectId	Reference to user
companyName	String	Company name
companyEmail	String	Company email
companyWebsite	String	Company website

4. Jobs Collection

Stores all job postings.

Field Name	Data Type	Description
jobId	ObjectId	Unique job ID
recruiterId	ObjectId	Job creator
title	String	Job title
description	String	Job details
location	String	Job location
salary	String	Salary package
createdAt	Date	Posting date

5. Applications Collection

Stores job application records.

Field Name	Data Type	Description
applicationId	ObjectId	Unique application ID
studentId	ObjectId	Applicant student
jobId	ObjectId	Applied job
status	String	Pending / Accepted / Rejected
appliedAt	Date	Application submission date

4.4 Relationships Between Entities

User → Student

One user account can belong to one student profile.

User → Recruiter

One user account can belong to one recruiter profile.

Recruiter → Jobs

One recruiter can create multiple job postings.

Student → Applications

One student can apply for multiple jobs.

Job → Applications

One job posting can receive multiple applications.

Admin → System Management

Admin has access to manage users, jobs, and applications throughout the system.

5. UML Diagrams

5.1 Use Case Diagram

The Use Case Diagram represents the interaction between system users and the UniHirex platform. It identifies the major functionalities provided by the system and the actors involved.

Actors

- Student
- Recruiter
- Admin

Student Use Cases

- Register Account
- Login
- Manage Profile
- Search Jobs
- Apply for Jobs

- Track Application Status

Recruiter Use Cases

- Register Account
- Login
- Post Jobs
- Edit/Delete Jobs
- View Applicants
- Manage Applications

Admin Use Cases

- Manage Users
- Manage Jobs
- Monitor System
- Generate Reports

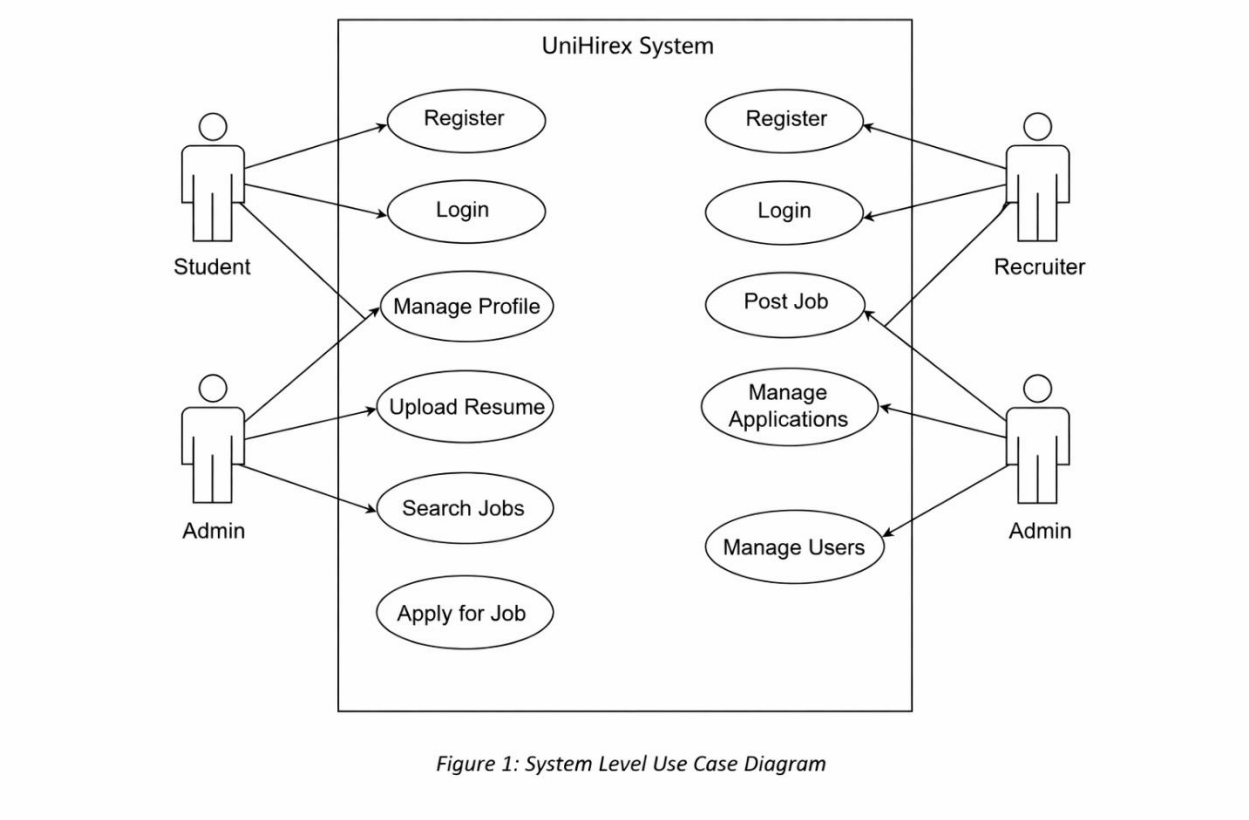


Figure 1: System Level Use Case Diagram

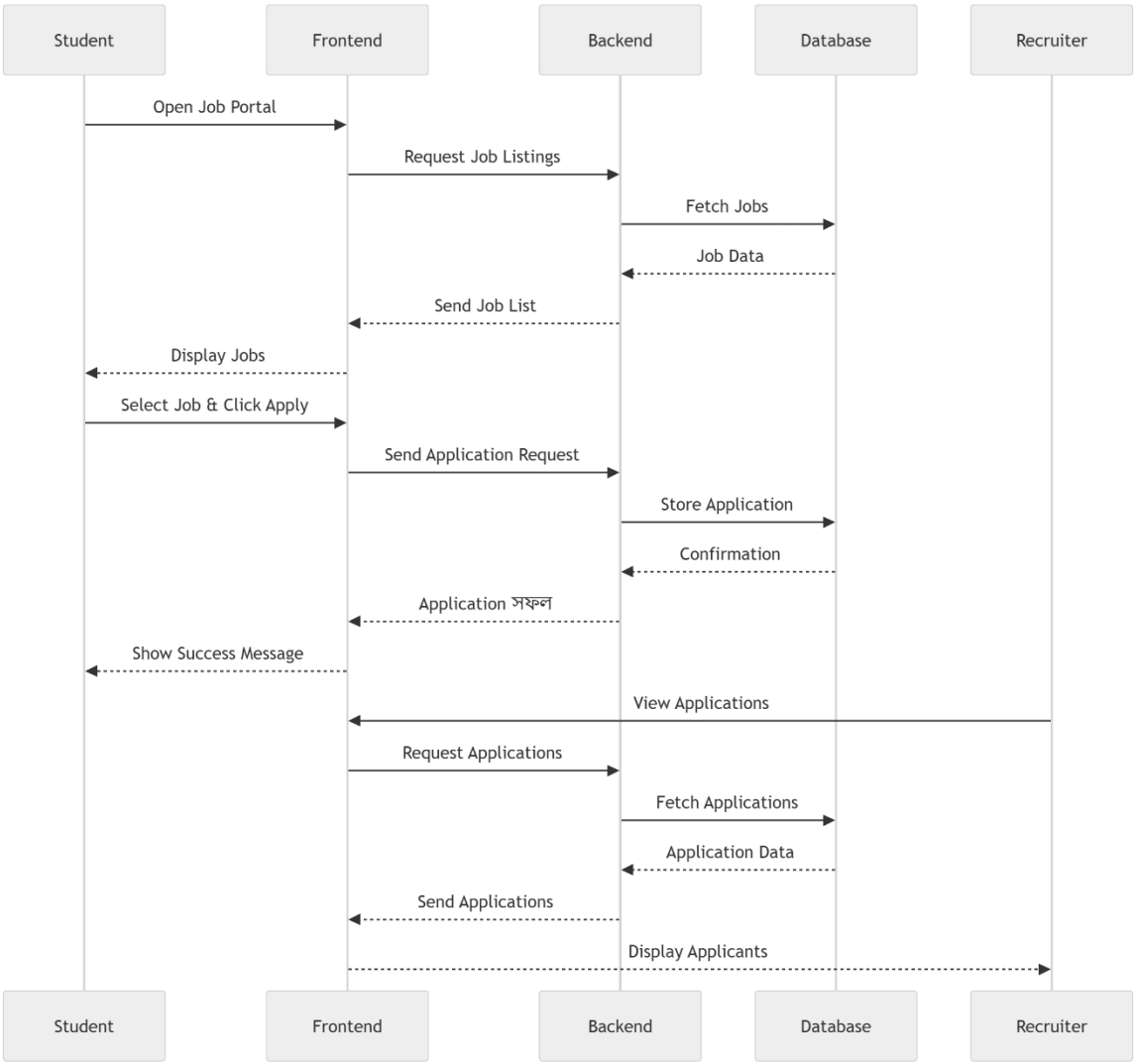
5.2 Sequence Diagram

The Sequence Diagram illustrates how objects interact with each other in a sequence to perform a specific function.

Example Flow: Student Applying for a Job

1. Student logs into the system.
2. Student searches for jobs.
3. System fetches job listings from database.
4. Student selects a job and submits application.
5. Backend validates request.
6. Application data is stored in database.
7. System sends confirmation response.

The sequence diagram helps visualize communication between frontend, backend, and database components.



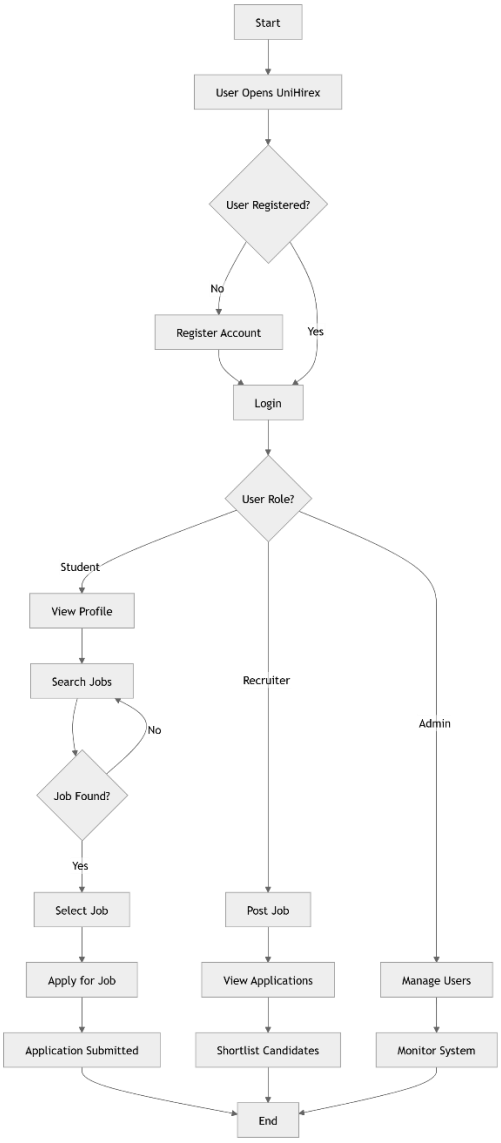
5.3 Activity Diagram

The Activity Diagram represents the workflow of system processes and user activities.

Main Activities

- User Registration
- User Authentication
- Job Posting
- Job Searching
- Job Application Submission
- Application Review
- Admin Monitoring

The diagram shows decision points, process flows, and parallel activities within the system.



5.4 Class Diagram

The Class Diagram represents the structure of the system by showing classes, attributes, methods, and relationships between them.

Main Classes

- User
- Student
- Recruiter
- Admin
- Job
- Application

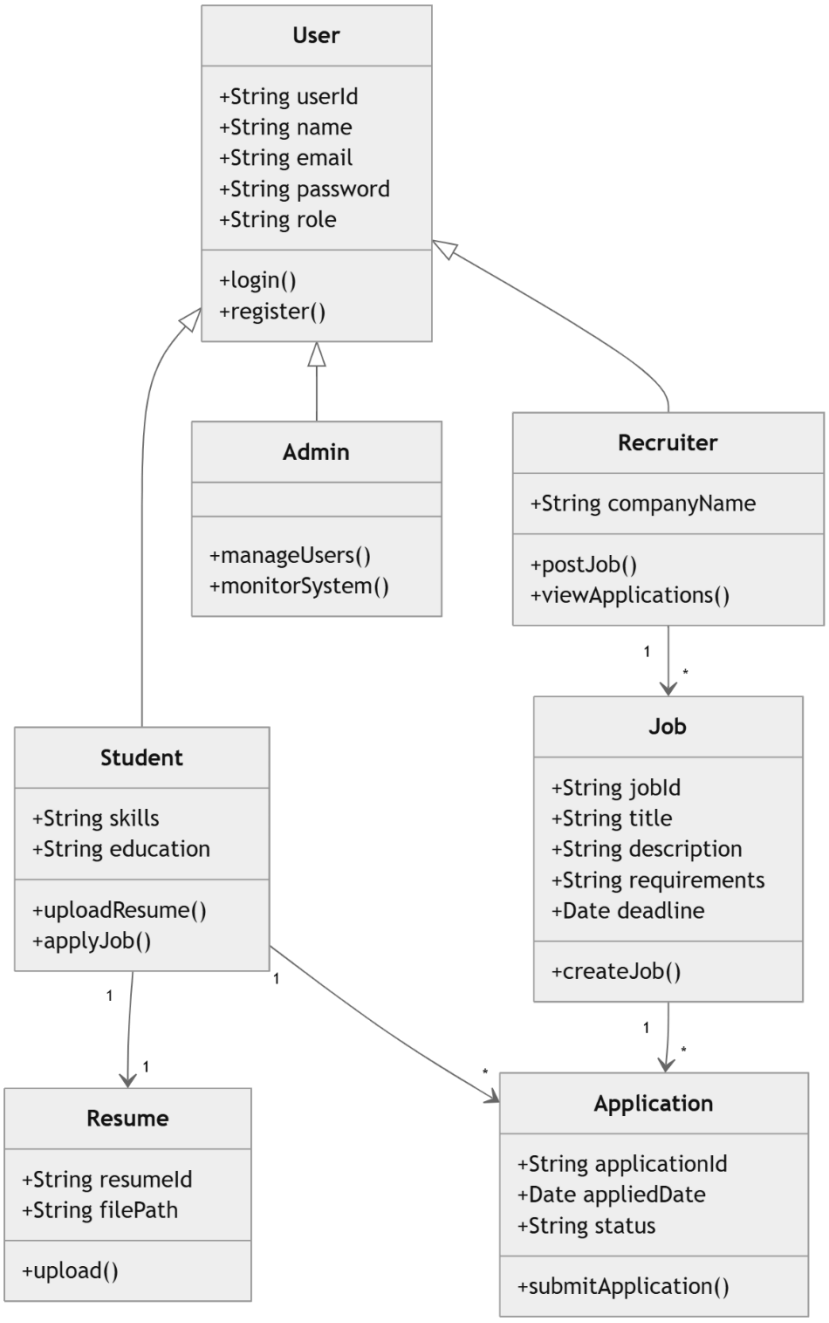
Example Relationships

- Student inherits from User
- Recruiter inherits from User
- Recruiter creates Jobs
- Student applies for Jobs
- Job contains multiple Applications

Common Attributes

- User: name, email, password, role
- Job: title, description, salary, location
- Application: status, appliedDate

The class diagram helps developers understand object-oriented design and system structure.



6. Module Design

6.1 Student Module

The Student Module is responsible for managing all functionalities related to students. This module allows students to create accounts, manage profiles, search for jobs, and apply for job opportunities.

Main Features

- Student Registration and Login
- Profile Management
- Resume Upload
- Job Search and Filtering
- Apply for Jobs
- Track Application Status

Inputs

- Registration details
- Login credentials
- Resume and profile data
- Job application information

Outputs

- Authentication confirmation
- Job listings
- Application status updates
- Notifications

This module is designed to provide a user-friendly experience for students and help them connect with recruiters efficiently.

6.2 Recruiter Module

The Recruiter Module handles all operations related to recruiters and companies. Recruiters can manage job postings and review applications submitted by students.

Main Features

- Recruiter Registration and Login
- Company Profile Management
- Create Job Posts
- Edit and Delete Jobs
- View Applicants
- Update Application Status

Inputs

- Company details
- Job information

- Applicant review actions

Outputs

- Job posting confirmations
- Applicant lists
- Application updates

This module helps recruiters efficiently manage recruitment activities through a centralized system.

6.3 Admin Module

The Admin Module is responsible for managing and monitoring the entire UniHirex system.

Main Features

- Manage Users
- Manage Job Listings
- Monitor System Activities
- Remove Unauthorized Content
- Generate Reports and Analytics

Inputs

- User management actions
- Job management actions
- Monitoring requests

Outputs

- User reports
- System analytics
- Activity logs

The admin module ensures platform security, stability, and proper system management.

6.4 Authentication Module

The Authentication Module handles user verification and secure access control within the system.

Main Features

- User Registration
- User Login
- Password Encryption
- JWT Token Generation
- Session Management
- Role-Based Access Control

Security Mechanisms

- Encrypted passwords using hashing algorithms
- Secure token-based authentication
- Protected API routes

This module ensures that only authorized users can access system resources.

6.5 Job and Application Management Module

This module manages all operations related to jobs and applications within the platform.

Job Management Features

- Create Job Listings
- Update Job Listings
- Delete Job Listings
- View Available Jobs

Application Management Features

- Submit Applications
- Store Application Records
- View Application Details
- Update Application Status

Data Managed

- Job Information
- Student Applications
- Application Status Records

This module serves as the core functional component of the UniHirex recruitment platform.

7. User Interface Design

7.1 Homepage Design

The homepage serves as the main entry point of the UniHirex platform. It is designed to provide a clean, modern, and user-friendly interface for students, recruiters, and administrators.

Main Components

- Navigation Bar
- Hero Section
- Featured Job Listings
- About Platform Section
- Call-to-Action Buttons
- Footer Section

Features

- Responsive design for mobile and desktop devices
- Easy navigation to login and registration pages
- Quick access to job listings and platform information

The homepage is designed to create a professional first impression and improve user engagement.

7.2 Login and Registration Pages

The login and registration pages allow users to securely access the platform and create accounts.

Registration Features

- Student Registration
- Recruiter Registration
- Form Validation
- Password Confirmation

Login Features

- Secure Authentication
- JWT-Based Login System
- Forgot Password Support

UI Design Elements

- Clean form layouts

- Input validation messages
- Responsive design
- User-friendly interface

These pages ensure secure and smooth user authentication.

7.3 Student Dashboard

The Student Dashboard provides students with access to all student-related functionalities.

Main Features

- View and Edit Profile
- Upload Resume
- Browse Jobs
- Apply for Jobs
- Track Application Status
- Notifications Section

Dashboard Components

- Sidebar Navigation
- Job Recommendation Area
- Application Tracking Panel
- Profile Management Section

The dashboard is designed to help students efficiently manage their career-related activities.

7.4 Recruiter Dashboard

The Recruiter Dashboard enables recruiters to manage job postings and applications.

Main Features

- Manage Company Profile
- Create Job Posts
- Edit/Delete Jobs
- View Applicants
- Manage Applications

Dashboard Components

- Job Management Panel
- Applicant Tracking Section

- Company Information Area
- Notifications Section

The recruiter dashboard provides an organized interface for recruitment management.

7.5 Admin Panel

The Admin Panel provides complete administrative control over the system.

Main Features

- Manage Users
- Manage Job Listings
- Monitor System Activities
- View Reports and Analytics
- Remove Unauthorized Content

Dashboard Components

- User Management Section
- Job Monitoring Area
- Reports and Analytics Dashboard
- System Activity Logs

The admin panel ensures effective system monitoring, management, and security.

8. API Design

8.1 Authentication APIs

The Authentication APIs handle user registration, login, and secure access management within the UniHirex system.

Register User

- **Method:** POST
- **Endpoint:** /api/auth/register
- **Description:** Registers a new student or recruiter account.

Login User

- **Method:** POST
- **Endpoint:** /api/auth/login

- **Description:** Authenticates user credentials and generates a JWT token.

Logout User

- **Method:** POST
- **Endpoint:** /api/auth/logout
- **Description:** Logs out the currently authenticated user.

Forgot Password

- **Method:** POST
- **Endpoint:** /api/auth/forgot-password
- **Description:** Sends password reset request.

8.2 User APIs

The User APIs manage user profiles and account-related operations.

Get User Profile

- **Method:** GET
- **Endpoint:** /api/users/profile
- **Description:** Fetches authenticated user profile information.

Update User Profile

- **Method:** PUT
- **Endpoint:** /api/users/profile/update
- **Description:** Updates user profile details.

Upload Resume

- **Method:** POST
- **Endpoint:** /api/users/upload-resume
- **Description:** Uploads student resume.

Get All Users (Admin)

- **Method:** GET
- **Endpoint:** /api/admin/users
- **Description:** Retrieves all registered users.

8.3 Job APIs

The Job APIs handle job posting and job management functionalities.

Create Job

- **Method:** POST
- **Endpoint:** /api/jobs/create
- **Description:** Allows recruiters to create new job postings.

Get All Jobs

- **Method:** GET
- **Endpoint:** /api/jobs
- **Description:** Retrieves all available job listings.

Get Single Job

- **Method:** GET
- **Endpoint:** /api/jobs/:id
- **Description:** Retrieves details of a specific job.

Update Job

- **Method:** PUT
- **Endpoint:** /api/jobs/update/:id
- **Description:** Updates existing job details.

Delete Job

- **Method:** DELETE
- **Endpoint:** /api/jobs/delete/:id
- **Description:** Deletes a job posting.

8.4 Application APIs

The Application APIs manage job applications submitted by students.

Apply for Job

- **Method:** POST
- **Endpoint:** /api/applications/apply/:jobId
- **Description:** Submits a job application.

Get Student Applications

- **Method:** GET
- **Endpoint:** /api/applications/student
- **Description:** Retrieves applications submitted by a student.

Get Job Applicants

- **Method:** GET
- **Endpoint:** /api/applications/job/:jobId
- **Description:** Retrieves applicants for a specific job.

Update Application Status

- **Method:** PUT
- **Endpoint:** /api/applications/status/:id
- **Description:** Updates application status (Accepted/Rejected).

Delete Application

- **Method:** DELETE
- **Endpoint:** /api/applications/delete/:id
- **Description:** Removes an application record.

9. Testing Strategy

9.1 Unit Testing

Unit testing is performed to verify that individual components and functions of the UniHirex system work correctly. Each module is tested independently to ensure accurate functionality.

Components Tested

- Authentication functions
- API controllers
- Database operations
- Form validation functions
- Job and application management logic

Objectives

- Detect coding errors early
- Ensure each function performs as expected
- Improve system reliability

Unit testing helps developers identify issues during the development phase before integrating modules into the complete system.

9.2 Integration Testing

Integration testing is conducted to verify that different modules of the system communicate and interact correctly with each other.

Modules Integrated

- Frontend and Backend
- Backend and Database
- Authentication and Authorization System
- Job Management and Application Modules

Objectives

- Ensure smooth data flow between modules
- Verify API communication
- Detect interface and communication issues

This testing ensures that combined system components function properly as a complete application.

9.3 System Testing

System testing is performed on the fully integrated UniHirex platform to ensure that the complete system meets the specified requirements.

Testing Areas

- Functional Testing
- Performance Testing
- Security Testing
- User Interface Testing
- Compatibility Testing

Objectives

- Verify overall system functionality
- Ensure system stability and reliability
- Validate performance under different conditions

System testing confirms that the application is ready for deployment and real-world usage.

9.4 User Acceptance Testing (UAT)

User Acceptance Testing is conducted to verify that the system satisfies user requirements and expectations.

Participants

- Students
- Recruiters
- Project Supervisor / Stakeholders

Testing Activities

- Registering accounts
- Posting jobs
- Applying for jobs
- Managing applications
- Navigating dashboards

Objectives

- Validate user requirements
- Ensure ease of use
- Collect feedback for improvements

UAT ensures that the final system is user-friendly, functional, and suitable for practical use.

10. Security Design

10.1 Authentication and Authorization

The UniHirex system implements secure authentication and authorization mechanisms to protect user accounts and system resources.

Authentication Features

- User registration and login system
- JWT (JSON Web Token) based authentication
- Secure session handling
- Role-based access control

Authorization Levels

- **Students:** Access student-related functionalities only
- **Recruiters:** Access recruitment and job management features
- **Admins:** Full system management access

Protected routes are implemented to ensure that only authorized users can access restricted resources.

10.2 Data Validation

Data validation is applied on both frontend and backend to ensure data accuracy and prevent malicious input.

Validation Mechanisms

- Required field validation
- Email format validation
- Password strength validation
- Input sanitization
- Duplicate data prevention

Benefits

- Improves data consistency
- Prevents invalid submissions
- Reduces security vulnerabilities

Proper validation ensures reliable and secure system operation.

10.3 Password Encryption

User passwords are securely encrypted before being stored in the database.

Encryption Techniques

- Password hashing using bcrypt
- Salt generation for enhanced security
- Secure password comparison during login

Security Benefits

- Prevents plain-text password storage
- Protects user credentials in case of data breaches
- Enhances overall account security

The system follows secure password management practices to ensure user data protection.

10.4 Secure API Communication

The UniHirex platform ensures secure communication between frontend and backend services.

Security Measures

- HTTPS-based communication
- JWT token verification
- Secure API endpoints
- Authorization middleware
- Error handling and response protection

Benefits

- Protects sensitive data during transmission
- Prevents unauthorized API access
- Ensures secure client-server communication

These measures help maintain system confidentiality, integrity, and reliability.

11. Performance and Scalability

11.1 System Performance

The UniHirex system is designed to provide fast, reliable, and efficient performance for students, recruiters, and administrators.

Performance Features

- Fast API response handling
- Optimized database queries
- Efficient frontend rendering using React.js
- Reduced page reloads through single-page application architecture
- Asynchronous request handling

Performance Objectives

- Minimize system response time
- Ensure smooth user experience
- Handle multiple user requests efficiently
- Maintain stable performance during high traffic

The system architecture is optimized to ensure consistent performance across different devices and network conditions.

11.2 Scalability Considerations

The UniHirex platform is designed with scalability in mind to support future growth in users, job postings, and universities.

Scalability Features

- Modular system architecture
- Cloud-based deployment support
- Scalable MongoDB database structure
- RESTful API architecture
- Separation of frontend and backend services

Future Scalability

The system can be expanded to support:

- Multiple universities
- Large numbers of concurrent users
- Additional recruitment features
- Mobile applications
- AI-based recommendation systems

The scalable architecture ensures that the system can handle increasing workloads without major structural changes.

12. Future Enhancements

12.1 AI-Based Job Recommendations

In future versions, the UniHirex platform can integrate Artificial Intelligence (AI) to provide personalized job recommendations for students based on their skills, qualifications, interests, and application history.

Benefits

- Improved job matching accuracy
- Better user experience
- Increased recruitment efficiency

12.2 Mobile Application

A mobile application can be developed for Android and iOS platforms to improve accessibility and user convenience.

Features

- Mobile-friendly interface
- Push notifications
- Real-time updates
- Easy job application process

This enhancement will allow users to access the platform anytime and anywhere.

12.3 Multi-University Support

Currently, the system is designed for a single university environment. In the future, the platform can be expanded to support multiple universities.

Possible Enhancements

- University-specific portals
- Separate admin management for each university
- Cross-university recruitment opportunities

This feature will increase the platform's reach and scalability.

12.4 Real-Time Notifications

The system can be upgraded with a real-time notification feature to improve communication between students and recruiters.

Notification Types

- Job application updates
- New job postings
- Interview invitations
- System announcements

Technologies such as WebSockets or Firebase notifications can be integrated for real-time communication.

12.5 Interview Scheduling System

An interview scheduling module can be added to streamline the recruitment process.

Features

- Schedule interviews directly through the platform
- Automatic email notifications
- Calendar integration
- Online interview meeting links

This enhancement will simplify communication and recruitment coordination between recruiters and students.

13. References

[1] MongoDB, “MongoDB Documentation,” MongoDB Inc. [Online]. Available: MongoDB Official Documentation. [Accessed: May 15, 2026].

[2] React, “React Documentation,” Meta Platforms Inc. [Online]. Available: React Official Documentation. [Accessed: May 15, 2026].

[3] Node.js, “Node.js Documentation,” OpenJS Foundation. [Online]. Available: Node.js Official Documentation. [Accessed: May 15, 2026].

[4] Express.js, “Express.js Guide,” Express.js. [Online]. Available: Express.js Official Documentation. [Accessed: May 15, 2026].

[5] GitHub, “GitHub Documentation,” GitHub Inc. [Online]. Available: GitHub Docs. [Accessed: May 15, 2026].

[6] Visual Studio Code, “Visual Studio Code Documentation,” Microsoft. [Online]. Available: VS Code Documentation. [Accessed: May 15, 2026].

[7] Postman, “Postman Learning Center,” Postman Inc. [Online]. Available: Postman Documentation. [Accessed: May 15, 2026].

[8] Software Engineering, I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[9] Database Management Systems, A. Silberschatz, H. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[10] Web Development, J. Duckett, *JavaScript and JQuery: Interactive Front-End Web Development*. Indianapolis, IN, USA: Wiley, 2014.